

Java Backend Architecture

Interview Series

Chapter 8.4

OpenID Connect (OIDC) & SSO

50+ Interview Q&A

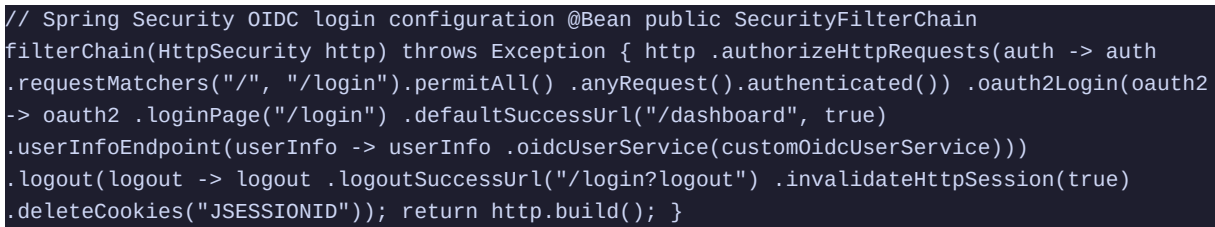
Code Examples

Architecture Diagrams

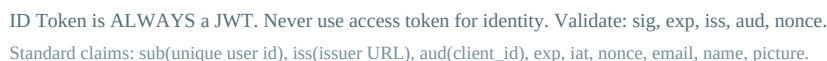
Advanced Topics

Introduction

OIDC Flow



ID Token Structure



Claim	Full Name	Description	Example
sub	Subject	Unique user ID at IdP (stable, never changes)	7f1f77bcf86cd799439011
iss	Issuer	IdP URL — must match expected issuer	https://accounts.google.com
aud	Audience	client_id — token intended for this client	my-app-client-id
exp	Expiration	Unix timestamp — reject if past this	1700000000
iat	Issued At	When token was issued	1699996400

nonce	Nonce	Replay prevention — must match what was sent	at=1699990000
email	Email	User email (if email scope granted)	user@example.com
name	Full Name	Display name (if profile scope)	Jane Doe
picture	Avatar URL	Profile picture URL	https://...
at_hash	Access Token Hash	Hash of access token (validates binding)	SpHBqJl...

UserInfo Endpoint

The UserInfo endpoint returns claims about the authenticated user. The client calls it with the access token (not ID token). Some claims are in the ID token (lightweight); additional claims require a UserInfo call. Spring Security automatically calls UserInfo if configured.

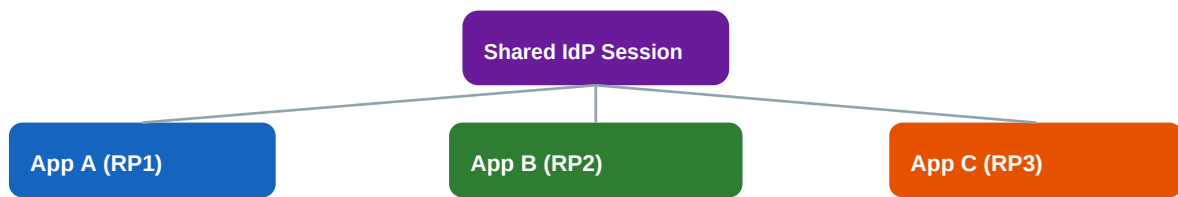
```
// GET /userinfo // Authorization: Bearer <access_token> // // Response: // { // "sub":
"auth0|507f1f77bcf86cd799439011", // "name": "Jane Doe", // "email": "jane@example.com", //
"email_verified": true, // "picture": "https://cdn.example.com/jane.jpg", // "locale":
"en-US", // "updated_at": 1699990000 // } // Custom OIDC user service in Spring @Service public
class CustomOidcUserService extends OidcUserService { @Override public OidcUser
loadUser(OidcUserRequest userRequest) throws OAuth2AuthenticationException { OidcUser oidcUser
= super.loadUser(userRequest); // Sync user to local DB, assign application roles String sub =
oidcUser.getSubject(); String email = oidcUser.getEmail(); userRepository.upsert(sub, email);
Set<GrantedAuthority> authorities = loadAuthorities(sub); return new
DefaultOidcUser(authorities, oidcUser.getIdToken(), oidcUser.getUserInfo()); } }
```

Discovery Document

Every OIDC provider exposes a discovery document at **/well-known/openid-configuration**. This JSON document lists all endpoints and supported features — clients use it for auto-configuration.

Key	Description
issuer	The IdP URL — must match id_token iss claim
authorization_endpoint	Where to send authorization requests
token_endpoint	Where to exchange code for tokens
userinfo_endpoint	Where to fetch user claims
jwks_uri	JSON Web Key Set — public keys for JWT verification
end_session_endpoint	Logout endpoint (RP-initiated logout)
scopes_supported	List of supported scopes: openid, profile, email, ...
response_types_supported	code, token, id_token
claims_supported	sub, iss, email, name, picture, ...
code_challenge_methods_supported	S256 (SHA256 PKCE — required)

SSO & Session Management



SSO: user logs in once at IdP. Each app checks IdP session. logout at IdP ends all app sessions (RP-initiated logout).

Logout Type	Mechanism	How It Works
RP-Initiated Logout	GET /end_session?id_token_hint=...&post_logout_redirect_uri=...	App logs out, IdP logs out, IdP ends session
Front-Channel Logout	IdP loads app logout URLs in iframes	IdP notifies each RP to clear local session via browser
Back-Channel Logout	IdP POSTs logout token to each RP	Server-to-server — more reliable, works without browser
Session Check Iframe	RP polls IdP iframe periodically	Detect IdP session changes without user action

Keycloak / Auth0 / Okta Integration

```
# application.yaml – Spring Boot with Keycloak as OIDC provider
spring:
  security:
    oauth2:
      client:
        registration:
          keycloak:
            client-id: my-spring-app
            client-secret: ${KEYCLOAK_CLIENT_SECRET}
            scope: openid,profile,email,roles
            authorization-grant-type: authorization_code
        provider: keycloak:
            issuer-uri: http://keycloak:8080/realms/my-realm # Spring auto-discovers all endpoints from /.well-known/openid-configuration
            # For resource server (validating JWTs from Keycloak):
            spring:
              security:
                oauth2:
                  resourceserver:
                    jwt:
                      issuer-uri: http://keycloak:8080/realms/my-realm # Spring fetches JWKS from Keycloak and caches it
```

50+ Interview Questions & Answers

Q1. What is OpenID Connect?

OIDC 1.0 is an identity layer on top of OAuth 2.0. It adds the ID Token (JWT with user identity claims), the UserInfo endpoint, and the discovery document. OAuth 2.0 handles authorization; OIDC handles authentication.

Q2. What is the difference between OAuth 2.0 and OIDC?

OAuth 2.0: grants access to resources via access tokens. OIDC: adds identity — who is the user — via ID tokens. OIDC uses OAuth 2.0 flows but adds scope=openid, ID token, UserInfo endpoint, and nonce for replay prevention.

Q3. What is an ID token?

A JWT issued by the IdP containing the user's identity claims: sub (unique user ID), iss, aud, exp, iat, and optionally name, email, picture. Must be validated: signature, exp, iss, aud, nonce.

Q4. What is the difference between ID token and access token?

ID token: for the client — proves who the user is (identity). Access token: for the resource server — grants access to APIs. Never send ID token to APIs. Never use access token to identify users (it may not contain user info).

Q5. What is the sub claim?

Subject — the unique, stable identifier for the user at the IdP. Never changes even if email changes. Use sub to identify users in your database (not email, which can be reassigned).

Q6. What is the nonce in OIDC?

A random value included in the authorization request. The IdP echoes it in the ID token. Client validates nonce matches — prevents ID token replay attacks. Required for implicit flow; recommended for code flow.

Q7. What is the UserInfo endpoint?

A protected resource at the IdP that returns additional claims about the user. Client calls it with the access token: GET /userinfo. Returns JSON with sub, name, email, picture, etc. More claims than in the ID token.

Q8. What is the OIDC discovery document?

JSON document at /.well-known/openid-configuration. Contains all IdP endpoint URLs, supported scopes, response types, signing algorithms. Spring Security uses issuer-uri to auto-discover all configuration.

Q9. What is SSO?

Single Sign-On — user authenticates once at a central IdP and gains access to multiple applications without re-authenticating. IdP maintains a session; each application checks the IdP session. Common implementations: Keycloak, Auth0, Okta, Azure AD.

Q10. How does OIDC enable SSO?

After login, IdP creates a session cookie. When user visits another app (RP), the app redirects to IdP's /authorize. IdP detects existing session, issues new tokens for that RP without prompting the user again (prompt=none).

Q11. What is RP-initiated logout?

The application (Relying Party) redirects the user to the IdP's end_session_endpoint. Parameters: id_token_hint (the ID token), post_logout_redirect_uri. IdP terminates its session, then redirects back. Ends SSO session across all apps.

Q12. What is front-channel logout?

IdP loads each registered RP's logout URL in invisible iframes in the browser. RPs detect this and clear their local sessions. Less reliable than back-channel (depends on browser, iframes can be blocked).

Q13. What is back-channel logout?

IdP sends an HTTP POST with a signed logout_token JWT to each RP's backchannel_logout_uri. RP validates the token and terminates the user's local session. More reliable than front-channel — no browser dependency.

Q14. What is prompt=none in OIDC?

A parameter that tells the IdP not to show any UI (login or consent). If the user has an active session, IdP silently issues tokens. If no session exists, returns error=login_required. Used for SSO silent token refresh.

Q15. What is prompt=login in OIDC?

Forces re-authentication even if the user has an active IdP session. Used for sensitive operations (password change, payment confirmation) where you want to ensure freshly authenticated identity.

Q16. What is the acr claim?

Authentication Context Class Reference — indicates the level of assurance for the authentication. Examples: urn:mace:incommon:iap:silver, urn:mace:incommon:iap:gold, or LOA levels. Used to require MFA for sensitive operations.

Q17. What is the amr claim?

Authentication Methods References — how the user was authenticated. Examples: pwd (password), otp (one-time password), mfa, hwk (hardware key). Client can verify MFA was performed by checking for 'mfa' or

'otp' in amr.

Q18. What is PKCE in OIDC flows?

OIDC uses the same OAuth 2.0 Authorization Code + PKCE flow. Mandatory for public clients. code_challenge prevents authorization code interception. Spring Security enables PKCE automatically for public client registrations.

Q19. What is id_token_hint?

An ID token passed as a hint to the IdP in authorization or logout requests. In logout: tells IdP which user is logging out and which client. In silent auth (prompt=none): helps IdP identify the expected user.

Q20. How does Spring Security store the OIDC user?

After successful login, Spring creates an OidcUser object containing OidcIdToken and OidcUserInfo. Stored in SecurityContext as OAuth2AuthenticationToken with principal being the OidcUser. Session stores the OAuth2AuthorizedClient.

Q21. What is OidcUserService?

Spring's default service for loading OIDC users. Validates ID token, calls UserInfo endpoint, merges claims. Override it to customize user loading: sync to DB, add application roles, transform claims.

Q22. What is OAuth2AuthorizedClientService?

Spring service that manages OAuth2AuthorizedClient objects — stores access tokens and refresh tokens per user per client registration. InMemoryOAuth2AuthorizedClientService (default) or JdbcOAuth2AuthorizedClientService (production).

Q23. What is the difference between session-based and OIDC-based SSO?

Session-based SSO: sticky sessions or shared session store (Redis) across apps — only works in same domain or controlled infrastructure. OIDC SSO: uses standard protocol, works across domains, different tech stacks, and organizations.

Q24. What is federated identity?

A user's identity from one organization (e.g., Google, Azure AD) is trusted and used by another application without the application managing its own user database. OIDC enables federation — app trusts the IdP's assertion of identity.

Q25. What is a Relying Party (RP) in OIDC?

The application that relies on the IdP to authenticate the user. Equivalent to the OAuth 2.0 client. Registers with the IdP, holds client_id and client_secret, validates ID tokens.

Q26. What is an OpenID Provider (OP)?

The server that authenticates the user and issues ID tokens. Examples: Keycloak, Auth0, Okta, Google Identity, Azure AD B2C. Exposes the OIDC endpoints defined in the discovery document.

Q27. What is Keycloak?

Open-source IAM platform. Features: multi-realm, client scopes, composite roles, user federation (LDAP/AD), social login, custom authenticators, admin REST API, token exchange, fine-grained authorization. Spring integrates via standard OIDC.

Q28. What is Azure AD B2C?

Microsoft's cloud IAM for customer-facing apps. Supports OIDC, OAuth 2.0, SAML. Features: custom user flows, custom policies (IEF), social identity providers, MFA, custom branding. Spring Boot integrates via spring-security-oauth2-client.

Q29. What is the at_hash claim?

Access token hash — the left half of SHA256(access_token) base64url-encoded. Included in ID token when access token is issued in the same response. Client can verify access token integrity by recomputing at_hash.

Q30. What is c_hash?

Code hash — SHA256 hash of the authorization code, included in ID token (hybrid flow). Lets client verify the code matches the ID token. Less common than at_hash; used in hybrid flow.

Q31. What is OIDC hybrid flow?

response_type=code+id_token or code+token — returns ID token and/or access token in the authorization response AND issues tokens via the token endpoint. Rarely used; code flow + PKCE is preferred.

Q32. What is claims parameter in OIDC?

JSON object specifying which claims are needed and whether they're essential or voluntary. Example: claims={'userinfo':{'email':{'essential':true}}}. IdP must include these claims in UserInfo or ID token.

Q33. What is max_age parameter?

OIDC parameter specifying the maximum elapsed time (seconds) since last authentication. If exceeded, IdP must re-authenticate. Similar to prompt=login but time-based. auth_time claim in ID token shows when authentication occurred.

Q34. What is login_hint?

Parameter suggesting the user's identifier to the IdP (e.g., email address). IdP pre-fills the login form. Doesn't bypass authentication — just improves UX.

Q35. What is the session_state parameter?

Keycloak-specific (not standard) — represents the IdP session state. Used in session management iframe to detect session changes without making network requests.

Q36. How do you handle token refresh in OIDC?

Use offline_access scope to get refresh token. When access token expires, call token endpoint with grant_type=refresh_token. Spring's OAuth2AuthorizedClientManager auto-refreshes tokens when calling downstream services.

Q37. What is OAuth2AuthorizedClientManager?

Spring abstraction for managing token lifecycle — obtaining, refreshing, and revoking authorized clients. Use with ServerOAuth2AuthorizedClientExchangeFilterFunction for WebClient to auto-attach and refresh tokens.

Q38. What is the principal name in Spring OIDC?

By default, the sub claim (unique user ID). Can be customized via OidcUserAuthority's PrincipalClaimName. Use sub (not email) as the stable identifier in your database.

Q39. What is the difference between OIDC and SAML?

OIDC: JSON/JWT-based, uses OAuth 2.0 flows, designed for modern web/mobile, better for APIs. SAML: XML-based, older standard, common in enterprise/legacy, uses browser redirects with XML assertions. Both support SSO.

Q40. How does OIDC handle MFA?

IdP enforces MFA during the authentication step. Application can require MFA by specifying acr_values in the request or checking acr/amr claims in the ID token. Application cannot bypass IdP's MFA requirements.

Q41. What is back-channel token revocation in OIDC?

When a `logout_token` is received at `backchannel_logout_uri`, the app must find all sessions for that sub (and optionally sid) and terminate them. Should be idempotent and return 200 OK even for already-terminated sessions.

Q42. What is the sid claim?

Session ID — unique identifier for the IdP authentication session. Present in ID token and logout token. Used in back-channel logout to identify which session to terminate (a user can have multiple concurrent sessions).

Q43. What is OIDC Conformance?

Certification that an OIDC provider correctly implements the specification. OpenID Foundation runs conformance test suites. Major IdPs (Google, Auth0, Keycloak) are certified. Check conformance before using an IdP in production.

Q44. How does token caching work in Spring?

`OAuth2AuthorizedClientService` caches tokens per (principalName, clientRegistrationId). `InMemoryOAuth2AuthorizedClientService` stores in a `ConcurrentHashMap`. `JdbcOAuth2AuthorizedClientService` persists to DB — required for multi-instance deployments.

Q45. What is OAuth 2.0 for Browser-Based Apps (BFF pattern)?

IETF draft recommends using a Backend For Frontend (BFF) to handle OAuth in SPAs. The BFF holds tokens server-side; SPA gets a session cookie. Eliminates token exposure in JavaScript — critical for XSS prevention.

Q46. What is the OIDC RP-Initiated Logout redirect flow?

1) App redirects to `end_session_endpoint` with `id_token_hint` and `post_logout_redirect_uri`. 2) IdP validates the token hint, shows logout confirmation (optional). 3) IdP terminates its session. 4) IdP redirects to `post_logout_redirect_uri`. 5) App clears local session.

Q47. How do you debug OIDC issues?

Enable Spring Security debug logging. Check JWT at jwt.io. Verify discovery document is accessible. Check clock skew (leeway). Verify `client_id` and `redirect_uri` match registration. Check IdP logs. Use Postman to test token endpoint directly.

Q48. What is `OidcLogoutSuccessHandler` in Spring?

Spring 6 handler that redirects to IdP's `end_session_endpoint` on logout. Configured with: `.logout(l -> l.logoutSuccessHandler(new OidcClientInitiatedLogoutSuccessHandler(clientRegistrationRepository)))`. Sends `id_token_hint` automatically.

Q49. What is `DelegatingOAuth2TokenValidator`?

Spring class composing multiple JWT validators. Used to add custom validation: `JwtClaimValidator` for checking specific claims, `JwtTimestampValidator` for `expiry+skew`, `JwtIssuerValidator`. Example: `new DelegatingOAuth2TokenValidator<>(validators)`.

Q50. What is `JwtDecoder` in Spring?

Interface for decoding and validating JWT tokens. `NimbusJwtDecoder` is the default implementation using Nimbus JOSE library. Configure with `jwt-set-uri` or public key. Used by resource server to validate incoming bearer tokens.

Q51. How do you extract roles from Keycloak JWT in Spring?

Keycloak puts roles in `realm_access.roles` and `resource_access.{client_id}.roles` claims. Implement `JwtAuthenticationConverter` to extract these and map to `GrantedAuthority`: `new SimpleGrantedAuthority('ROLE_' + role)`.